

Lenguajes de Programación y Transducción

Terra: lenguaje de dominio específico para datos ambientales

Segunda entrega — Semántica y procesamiento de datos

Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 20 de octubre de 2026

Índice

1. Introducción **4**

1.1. Objetivos de la fase 4

1.2. Resumen de lo realizado 5

2. Arquitectura **5**

2.1. Separación en módulos 5

2.2. Cómo viajan los errores 6

3. Cambios en la gramática **6**

4. La estructura Tabla y el lector de CSV **7**

4.1. La clase Tabla 7

4.2. Lectura: un autómata de dos estados 7

4.3. Inferencia de tipos 7

4.4. Validaciones del lector 8

4.5. Escritura 8

5. Semántica de las instrucciones de datos **8**

5.1. cargar y guardar 8

5.2. Tuberías 8

5.3. Evaluar una expresión en cada fila 8

5.4. Agrupar y resumir 9

5.5. describir 9

6. Ampliaciones del lenguaje **9**

6.1. Matemáticas y matrices 9

6.2. Funciones anónimas y de orden superior 10

6.3. Gestor de memoria y límite de recursión 10

6.4. Entrada, tablas y archivos 11

6.5. Ayuda integrada 11

7. Programas de ejemplo y evidencia **11**

7.1. Transformación de datos 12

7.2. Agrupamiento 12

7.3. Evidencia de exportación 12

8. Caso de estudio con datos reales **13**

8.1. Validación cruzada 13

8.2. Rendimiento 14

9. Pruebas **14**

10. Decisiones de diseño **15**

11. Cumplimiento de los entregables **16**

12. Limitaciones y trabajo siguiente **16**

13. Reproducción **16**

1. Introducción

En la primera entrega construimos el *front-end* de Terra: la gramática en ANTLR, el análisis léxico y sintáctico, los mensajes de error en español y un evaluador para variables, expresiones, control de flujo y funciones. Las instrucciones propias del dominio (`cargar`, las tuberías `|>` y `guardar`) ya se reconocían en la gramática, pero al ejecutarlas el intérprete respondía que su ejecución correspondía al segundo corte.

En esta entrega esas instrucciones **se ejecutan**. Terra ya puede leer un archivo CSV, limpiarlo, transformarlo, agruparlo, calcular estadísticas y guardar el resultado, todo con su propia sintaxis y sin usar librerías externas de datos.

1.1. Objetivos de la fase

Según el enunciado del proyecto, la segunda fase debía:

- Implementar el patrón Visitor en Python y diseñar el contexto de ejecución y la tabla de símbolos.
- Asociar identificadores con objetos de datos.
- Implementar carga, selección, filtrado, ordenamiento, columnas calculadas y tratamiento de datos faltantes.
- Incorporar agrupamiento, agregaciones, estadísticas descriptivas y escritura de resultados en CSV.
- Validar los errores por variables o columnas inexistentes, tipos incompatibles y operaciones no permitidas.
- Demostrar que el lenguaje procesa **datos reales** y produce una tabla resumen correctamente exportada.

1.2. Resumen de lo realizado

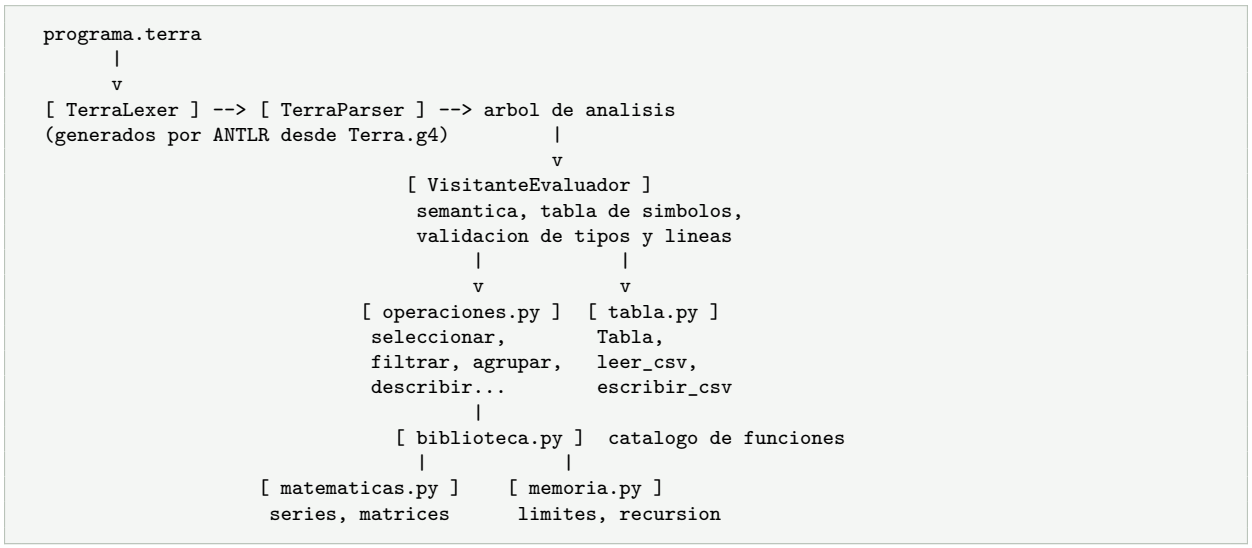
Componente	Qué se hizo
`src/tabla.py`	Nuevo. Estructura `Tabla`, lector y escritor de CSV escritos desde cero.
`src/operaciones.py`	Nuevo. Las once operaciones de datos, el agrupamiento y `describir`.
`src/visitante_evaluador.py`	Ejecuta `cargar`, `guardar` y las tuberías; agrega `contar` y `describir`.
`gramaticas/Terra.g4`	Opciones de carga y de guardado, funciones anónimas (`funcion(x) => ...`).
`src/biblioteca.py`	Nuevo. Catálogo de funciones incorporadas, funciones de orden superior y `ayuda()`.
`src/matematicas.py`	Nuevo. Trigonometría, exponencial y logaritmos por series de Taylor; vectores y matrices.
`src/memoria.py`	Nuevo. Gestor de memoria y límite de recursión.
`tests/`	Dos archivos nuevos con 107 pruebas; el proyecto pasa de 100 a 207.
`ejemplos/`	Once programas nuevos, incluido un caso de estudio con datos reales.
`salidas/`	Archivos CSV generados como evidencia de exportación.

La regla que se impuso para todo el corte es la misma del enunciado: **no usar ninguna librería**. No se usa pandas, NumPy ni siquiera el módulo `csv` de Python. La única dependencia sigue siendo el *runtime* de ANTLR.

2. Arquitectura

2.1. Separación en módulos

El trabajo se dividió para que cada módulo tenga una sola responsabilidad:



- **`tabla.py`** solo sabe representar datos y leer o escribir archivos.
- **`operaciones.py`** manipula tablas, pero *no conoce el árbol de ANTLR*. Cuando una operación depende de una expresión del lenguaje (por ejemplo `filtrar`), recibe una función que el evaluador le construye.
- **`visitante_evaluador.py`** es el único que entiende el árbol. Lee cada nodo, valida tipos y llama a las operaciones.
- **`biblioteca.py`** registra las funciones incorporadas con su nombre y número de argumentos. El evaluador se conecta a ella una sola vez, así que agregar una función nueva no requiere tocar la gramática ni el evaluador.

> **Por qué:** Si mañana cambiara la sintaxis de `filtrar`, solo habría que tocar la gramática y el evaluador; la lógica de filtrar filas no cambia. Y las operaciones de datos se pueden probar directamente, sin escribir un programa Terra.

2.2. Cómo viajan los errores

Los módulos de datos lanzan `ErrorDatos`, que no sabe en qué línea del programa está. El Visitor lo atrapa y lo convierte en `ErrorSemantico`, agregando la línea de la instrucción. Por eso todos los mensajes salen con el mismo formato:

```
Error en evaluacion: [linea 1] no se encontro el archivo 'datos/no_existe.csv'
```

3. Cambios en la gramática

La regla de carga solo admitía un separador opcional. El enunciado pide configurar también el encabezado, así que la regla pasó a aceptar una lista de opciones:

```
// Antes
carga
    : CARGAR expr (CON SEPARADOR expr)?
    ;

// Ahora
carga
    : CARGAR expr (CON opcion_carga)*
    ;

opcion_carga
    : SEPARADOR expr
    | ENCABEZADO expr
    ;

ENCABEZADO : 'encabezado';
```

Con esto se puede escribir:

```
fijo d = cargar "datos.csv" con separador ";" con encabezado falso
```

La gramática acepta las opciones en cualquier orden y repetidas; que una opción no se repita lo valida la semántica, que puede dar un mensaje más claro que un error sintáctico.

> **Por qué:** Se eligió `con encabezado falso` en lugar de `sin encabezado` para no convertir `sin` en palabra reservada. En un lenguaje con enfoque matemático, `sin` es el nombre natural de la función seno.

Después del cambio se regeneraron el lexer, el parser y el visitor con `./generar.sh`. Las 100 pruebas del primer corte siguieron pasando, lo que confirma que el cambio no rompió nada de la sintaxis anterior.

4. La estructura Tabla y el lector de CSV

4.1. La clase Tabla

Una tabla guarda una lista de nombres de columnas y una lista de filas, donde cada fila es una lista de valores alineada con las columnas. Al crearla se verifica que no haya columnas repetidas y que todas las filas tengan el número correcto de valores. Ofrece consultas como `columna(nombre)`, `tipo_columna(nombre)` y `como_texto()` para imprimirla alineada.

4.2. Lectura: un autómata de dos estados

Partir cada línea por comas no basta, porque un campo entre comillas puede contener comas, comillas escapadas e incluso saltos de línea:

```
nombre,nota
"Perez, Ana",4.5
"dijo ""hola""",3.0
```

Por eso el lector recorre el archivo carácter por carácter con un autómata de dos estados, *dentro* y *fuera* de comillas. Fuera de comillas, la coma corta el campo y el salto de línea corta la fila; dentro, son texto normal. Esto es la misma idea de los autómatas finitos vistos en el curso, aplicada a un formato de archivo.

Optimización. Si el archivo no tiene ninguna comilla, no puede haber campos protegidos, así que cada línea se parte directamente por el separador. Comprobamos con el archivo real de Our World in Data que los dos caminos dan exactamente el mismo resultado; el atajo es unas seis veces más rápido.

4.3. Inferencia de tipos

Los CSV solo tienen texto, así que hay que decidir el tipo de cada columna. Terra lo decide **por columna completa**, probando en orden booleano, número, decimal y texto. Las celdas vacías o con `NA`, `null` o `nulo` quedan como `nulo`.

Con el archivo de ejemplo el resultado es:

```
pais -> texto
anio -> numero
co2 -> decimal
poblacion -> numero
energia_renovable -> decimal
```

4.4. Validaciones del lector

El lector detecta archivos inexistentes, vacíos, con columnas sin nombre o repetidas, con filas que tienen más o menos campos que el encabezado (indicando la línea del archivo) y con comillas sin cerrar.

También detecta un error muy común: elegir el separador equivocado. Si cada fila queda como un solo campo y ese campo contiene otro separador conocido, Terra avisa:

```
Error en evaluacion: [linea 1] el archivo parece usar ',' como separador, no ';'. Revisa la opcion 'con
    separador'
```

Sin esta regla, Terra habría cargado una tabla de una sola columna sin quejarse, y el error habría aparecido varias líneas más adelante con un mensaje confuso.

4.5. Escritura

`escribir_csv` hace el proceso inverso: escribe el encabezado y las filas, deja los nulos como celdas vacías y encierra entre comillas los campos que lo necesitan. Una prueba automática verifica que guardar una tabla y volver a cargarla produce exactamente la misma tabla.

5. Semántica de las instrucciones de datos

El detalle de cada regla semántica está en el documento complementario *Documentación de las reglas semánticas*. Aquí se explica cómo se implementaron las ideas principales.

5.1. cargar y guardar

`visitCarga` evalúa la ruta, recorre las opciones verificando sus tipos y que no se repitan, y llama a `leer_csv`. `visitGuardar_stmt` verifica que el valor sea una tabla y la ruta un texto, y llama a `escribir_csv`. Una variable puede guardar una tabla igual que guarda un número, así que la tabla de símbolos del primer corte quedó asociando identificadores con objetos de datos sin necesidad de cambiar su estructura.

5.2. Tuberías

`visitTuberia` evalúa la expresión inicial, verifica que sea una tabla y aplica las operaciones en orden. Cada operación devuelve una **tabla nueva**:

```
fijo datos = cargar "datos/emisiones_ejemplo.csv"
fijo limpios = datos |> eliminar nulos
mostrar(datos)      # sigue teniendo sus 16 filas
```

5.3. Evaluar una expresión en cada fila

En `filtrar donde co2 > 50`, el nombre `co2` no es una variable del programa: es una columna. Para evaluarlo, el Visitor hace lo siguiente en cada fila:

1. Guarda una copia de lo que había en la memoria con esos nombres.

2. Declara las columnas como variables temporales, con el valor de la fila.
3. Evalúa la expresión con el mismo código que cualquier otra expresión de Terra.
4. Restaura la memoria, aunque haya ocurrido un error.

Así `filtrar` y `crear` reutilizan todos los operadores del lenguaje sin escribir un evaluador aparte.

Optimización. La primera versión declaraba todas las columnas en cada fila. Con el archivo real (79 columnas y 50 411 filas) eso son casi cuatro millones de asignaciones por cada `filtrar`, aunque la condición use una sola columna. Se agregó `_nombres_usados`, que recorre una sola vez el subárbol de la expresión, guarda los identificadores que aparecen en ella y solo declara esas columnas.

5.4. Agrupar y resumir

`agrupar por [pais]` recorre las filas y reúne en un diccionario los índices de las filas que comparten la misma clave. El resultado es un valor intermedio que solo admite `resumir` a continuación.

Para resumir se usó la misma técnica de la evaluación por fila, pero con una diferencia: en cada grupo, cada columna se declara como **la lista de sus valores no nulos**. Así, en

```
|> resumir total = suma(co2), promedio = media(co2)
```

`co2` es una lista y `suma` y `media` son las mismas funciones incorporadas del primer corte. No hubo que duplicar ninguna estadística. Solo se agregó `contar()`, que devuelve el número de filas del grupo.

5.5. describir

`describir(tabla)` devuelve otra tabla con una fila por columna. Para las numéricas calcula conteo, nulos, media, desviación estándar muestral, mínimo, mediana y máximo; para las de texto, valores distintos y el más frecuente. Las fórmulas están implementadas a mano en `operaciones.py`.

6. Ampliaciones del lenguaje

Además de lo que pide la fase, se compararon las capacidades de Terra con otro DSL de referencia del curso y se agregaron las que faltaban, adaptadas al dominio ambiental.

6.1. Matemáticas y matrices

`src/matematicas.py` implementa sin el módulo `math`:

- **Seno y coseno** con su serie de Taylor, después de llevar el ángulo al intervalo $[-\pi, \pi]$ para que la serie converja en pocos términos:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

- **Exponencial** con la serie $e^x = \sum x^n/n!$, usando $e^x = (e^{x/2^k})^{2^k}$ para que x sea pequeño.

- **Logaritmo natural** escribiendo $x = m \cdot 2^k$ con $m \in [0.5, 1)$, de modo que $\ln x = k \ln 2 + \ln m$, y $\ln m$ con la serie de $2 \operatorname{atanh} \frac{m-1}{m+1}$.
- **Matrices:** producto, suma, transpuesta, traza, **determinante por eliminación de Gauss** con pivoteo parcial e **inversa por Gauss-Jordan**.
- Un **generador de números aleatorios** congruencial lineal con semilla, para que los programas sean reproducibles.

Para validar las series se compararon `seno`, `coseno`, `exp` y `ln` contra 20 000 valores aleatorios; todos coincidieron con un error menor a 10^{-9}.

```
det(A) = 10.0
inversa(A) = [[0.6, -0.7], [-0.2, 0.4]]
A x inversa(A) = [[1.0, 0.0], [0.0, 1.0]]
```

6.2. Funciones anónimas y de orden superior

Se agregó a la gramática una alternativa de `primario`:

```
funcion_anonima
    : FUNCION '(' (IDENTIFICADOR (',' IDENTIFICADOR)*)? ')' FLECHA_FUNCION expr
    ;
FLECHA_FUNCION : '=>';
```

Una función anónima, o el nombre de una función escrito sin paréntesis, es ahora un **valor** que se puede guardar en una variable o pasar a otra función:

```
fijo doble = funcion(x) => x * 2
mapear(numeros, doble)                          # [2, 4, ..., 20]
filtro(numeros, funcion(n) => n % 2 == 0)       # [2, 4, 6, 8, 10]
reducir(numeros, funcion(a, b) => a + b)        # 55
mapear(consumos, funcion(kwh) => kwh * factor)  # usa una variable del programa
```

Se agregaron diez funciones de orden superior: `mapear`, `filtro`, `reducir`, `ordenar_por`, `todos`, `alguno`, `componer`, `aplicar`, `aplicar_matriz` y `rango`.

6.3. Gestor de memoria y límite de recursión

Cada llamada a una función de Terra recorre muchas reglas de la gramática y usa varios niveles de la pila de Python. Sin control, una recursión profunda terminaba en un `RecursionError` de Python, en inglés y sin línea. `src/memoria.py` ajusta la pila de Python para que el límite de Terra (200 llamadas, configurable hasta 2000) se alcance primero:

```
Error en evaluacion: [linea 2] la funcion 'infinita' supero el limite de 200 llamadas anidadas; revisa el
    caso base de la recursion
```

Además, `memoria_info()` devuelve una tabla con cada variable y un estimado propio de la memoria que ocupa, `liberar("x")` elimina una variable, y las matrices se rechazan antes de crearse si pasan de 5 millones de celdas.

6.4. Entrada, tablas y archivos

- `leer` y `leer_numero` piden datos por teclado; la segunda vuelve a preguntar si lo escrito no es un número.
- `columna(t, "co2")` convierte una columna en lista, lo que conecta las tablas con las funciones de listas, de orden superior y de matrices.
- `unir` (interna e izquierda) y `concatenar` combinan tablas; `a_matriz` y `tabla_desde` pasan de tabla a matriz y al revés.
- Archivos de texto: leer, escribir, agregar, consultar y borrar.
- `guardar` acepta `con separador`, y los textos aceptan las secuencias de escape `\n` y `\t`.

6.5. Ayuda integrada

`ayuda()` muestra, dentro del mismo intérprete, cómo se escribe cada instrucción con un ejemplo, organizadas en ocho categorías.

```
terra> ayuda("mapear")
[funciones]
  mapear(lista, f)
      aplica f a cada elemento
      ejemplo: mapear([1, 2, 3], funcion(x) => x * 2)
```

7. Programas de ejemplo y evidencia

Se agregaron once programas completos. Todos se ejecutan con `python src/terra.py ejemplos/<archivo>`.

Programa	Qué demuestra
`07_carga_guardado.terra`	Cargar, mostrar y guardar una tabla.
`08_tuberias.terra`	Limpieza, columnas calculadas, filtrado, selección, orden, renombrado, conversión y límite.
`10_agrupacion.terra`	Agrupamiento con las siete agregaciones y resumen sin agrupar.
`11_describir.terra`	Resumen descriptivo de variables numéricas y categóricas.
`12_opciones_carga.terra`	Archivo sin encabezado y separado por punto y coma.
`13_caso_owid.terra`	Caso de estudio completo con datos reales.
`14_matematicas.terra`	Trigonometría, logaritmos y álgebra lineal.
`15_memoria_recursion.terra`	Recursión, consulta y liberación de memoria.
`16_orden_superior.terra`	Funciones anónimas y de orden superior.
`17_entrada.terra`	Calculadora de huella de carbono por teclado.
`18_tablas_archivos.terra`	Unir y concatenar tablas, matrices desde tablas y archivos.

7.1. Transformación de datos

```
fijo per_capita = datos
    |> eliminar nulos
    |> crear co2_per_capita = co2 * 1000000 / poblacion
    |> filtrar donde anio >= 2018
    |> seleccionar [pais, anio, co2, co2_per_capita]
    |> ordenar por co2_per_capita desc
```

```
pais     | anio |  co2 | co2_per_capita
---------+------+------+---------------
Chile    | 2018 | 88.4 |         4.7273
Chile    | 2021 | 83.6 |         4.3542
Ecuador  | 2018 | 41.2 |         2.4235
...
[10 filas x 4 columnas]
```

Verificación manual: $88.4 \times 1\,000\,000 / 18\,700\,000 = 4.7273$.

7.2. Agrupamiento

```
fijo por_pais = datos
    |> agrupar por [pais]
    |> resumir co2_total = suma(co2),
                co2_promedio = redondear(media(co2), 2),
                co2_mediana = mediana(co2),
                co2_desv = redondear(desviacion(co2), 2),
                registros = contar()
    |> ordenar por co2_total desc
```

```
pais     | co2_total | co2_promedio | co2_mediana | co2_desv | registros
---------+-----------+--------------+-------------+----------+----------
Colombia |     346.6 |        86.65 |       87.15 |     5.27 |         4
Chile    |     336.3 |        84.08 |        84.4 |     3.87 |         4
Peru     |     216.2 |        54.05 |          54 |     3.37 |         4
Ecuador  |     156.3 |        39.08 |        39.1 |     1.79 |         4
[4 filas x 8 columnas]
```

Verificación manual para Colombia: la suma es $86.2 + 92.5 + 79.8 + 88.1 = 346.6$ y la mediana es $(86.2 + 88.1)/2 = 87.15$.

7.3. Evidencia de exportación

Los programas generan estos archivos en la carpeta `salidas/`:

- A partir del conjunto de ejemplo: `copia_emisiones.csv`, `co2_per_capita.csv`, `resumen_por_pais.csv` y `descripcion_emisiones.csv`.
- A partir de los datos reales: `owid_sudamerica_por_pais.csv`, `owid_colombia.csv` y `owid_top_2023.csv`.

8. Caso de estudio con datos reales

Para cumplir el producto de la fase (*el DSL procesa datos reales y produce una tabla resumen correctamente exportada*) se usó el conjunto público de emisiones de CO_2 de **Our World in Data**: 50 411 filas, 79 columnas y 254 países y regiones, con datos hasta 2024.

El conjunto mezcla países con agregados como "World" o "South America". Los agregados no tienen código ISO, así que el programa los descarta con `eliminar nulos en [iso_code]`.

```
fijo owid = cargar "datos/owid-co2-data.csv"

fijo paises = owid
    |> eliminar nulos en [iso_code]
    |> seleccionar [country, year, population, co2, co2_per_capita]
    |> renombrar country como pais
    |> renombrar year como anio
    |> renombrar population como poblacion

fijo por_pais = paises
    |> filtrar donde pais en ["Argentina", "Brazil", "Chile", "Colombia", "Ecuador", "
    Peru"]
    |> filtrar donde anio >= 2000
    |> eliminar nulos en [co2, co2_per_capita]
    |> agrupar por [pais]
    |> resumir co2_acumulado = redondear(suma(co2), 1),
               co2_promedio = redondear(media(co2), 2),
               per_capita_promedio = redondear(media(co2_per_capita), 3),
               per_capita_maximo = maximo(co2_per_capita),
               anios = contar()
    |> ordenar por co2_acumulado desc

guardar por_pais como "salidas/owid_sudamerica_por_pais.csv"
```

```
pais      | co2_acumulado | co2_promedio | per_capita_promedio | per_capita_maximo | anios
----------+---------------+--------------+---------------------+-------------------+------
Brazil    |       11016.3 |       440.65 |               2.242 |             2.781 |    25
Argentina |        4336.4 |       173.46 |               4.128 |             4.641 |    25
Colombia  |        1933.3 |        77.33 |               1.672 |             2.027 |    25
Chile     |        1833.5 |        73.34 |               4.139 |             4.791 |    25
Peru      |        1103.2 |        44.13 |               1.444 |             2.053 |    25
Ecuador   |         882.4 |         35.3 |               2.257 |             2.729 |    25
[6 filas x 6 columnas]
```

El CO_2 se expresa en millones de toneladas y el CO_2 per cápita en toneladas por persona. Cada país tiene 25 años de datos (2000 a 2024), lo que coincide con las 150 filas que reporta `describir` para la región.

8.1. Validación cruzada

El programa también calcula, para Colombia, una columna propia:

```
|> crear co2_por_millon_hab = redondear(co2 / poblacion * 1000000, 3)
```

El resultado coincide exactamente con la columna `co2_per_capita` que publica Our World in Data (por ejemplo, 1.685 en 2010). Tiene sentido: millones de toneladas divididas entre millones de habitantes dan toneladas por persona. Esto es una comprobación independiente de que la lectura de decimales, la aritmética y `crear` funcionan correctamente sobre datos reales.

8.2. Rendimiento

Medición	Tiempo
Cargar el archivo completo (50 411 × 79)	1,7 s
Ejecutar el caso de estudio completo	5,2 s

Para un intérprete escrito en Python, recorriendo un árbol de ANTLR y sin librerías, son tiempos razonables. Se lograron con las dos optimizaciones descritas: el atajo del lector para archivos sin comillas y la declaración solo de las columnas usadas en cada expresión.

9. Pruebas

Las pruebas nuevas están en `tests/test_corte2.py` (datos) y `tests/test_biblioteca.py` (ampliaciones), y se ejecutan con `pytest`. Se organizaron en dos niveles:

- **Pruebas unitarias** de `tabla.py`: llaman directamente al lector y al escritor, sin pasar por el lenguaje.
- **Pruebas de integración**: ejecutan fragmentos de programa Terra a través de toda la cadena (lexer, parser, Visitor, operaciones y tabla) y verifican el valor resultante o el error esperado.

Grupo	Pruebas	Qué verifica
Lector y escritor CSV	10	Tipos, nulos, comillas, comillas escapadas, sin encabezado, filas mal formadas, separador equivocado, ida y vuelta.
Cargar y guardar	8	Opciones, tipos de las opciones, opción repetida, archivo inexistente, guardar algo que no es tabla.
Operaciones de tubería	22	Cada operación, sus errores y que la tabla original no cambie.
Agrupar y resumir	10	Las siete agregaciones, nulos ignorados, varias claves y las reglas de `agrupar`.
`describir` y `mostrar`	3	Estadísticas por tipo de columna y formato de salida.
Matemáticas y matrices	18	Valores conocidos, dominios inválidos, dimensiones, matriz singular, inversa por matriz igual a identidad.
Memoria y recursión	8	Recursión de 150 niveles, recursión infinita, límite configurable, matriz demasiado grande, liberar.
Orden superior	13	Las diez funciones, funciones con nombre como valor y variables del programa dentro de una anónima.
Entrada por teclado	3	Lectura simulada, reintento ante un dato no numérico y fin de la entrada.
Tablas y archivos	12	Unir, concatenar, matrices desde tablas, archivos de texto, separador al guardar y secuencias de escape.
Total del corte	**107**	

Cerca de la mitad son **pruebas negativas**: verifican que un programa incorrecto falle con el error correcto. Los valores esperados de las pruebas de estadística se calcularon a mano.

```
$ pytest tests/ -q
.....................................................................
.....................................................................
.......
207 passed
```

Las 100 pruebas del primer corte siguieron pasando después de cada cambio, lo que sirve como prueba de regresión.

10. Decisiones de diseño

Tablas inmutables.
Cada operación devuelve una tabla nueva. Ningún paso altera en silencio un resultado anterior.

Nulos estrictos en operaciones, ignorados en agregaciones.
Sumar un nulo es un error, para que los datos faltantes no pasen desapercibidos. En cambio `media` ignora los nulos, como en estadística descriptiva.

Inferencia por columna.
Una columna nunca queda con números en unas filas y textos en otras.

Condiciones estrictamente booleanas.
`filtrar donde co2` es un error, no se interpreta como "distinto de cero".

Reutilizar las funciones del lenguaje.
Las columnas se exponen como variables (en `filtrar`) o como listas (en `resumir`), así no hay un segundo evaluador de expresiones ni estadísticas duplicadas.

`con encabezado falso` en lugar de `sin encabezado`.
Para no reservar la palabra `sin`.

Catálogo de funciones aparte.
Las funciones nuevas se registran en `biblioteca.py`; el evaluador no cambia al agregar una.

Funciones como valores sin palabras nuevas.
`funcion(x) => ...` reutiliza la palabra reservada `funcion`.

Errores con línea y sugerencia.
Una columna inexistente lista las columnas disponibles, y un separador equivocado indica cuál parece ser el correcto.

11. Cumplimiento de los entregables

Entregable del enunciado	Dónde está
Gramática actualizada	`gramaticas/Terra.g4` (`opcion_carga`, separador en `guardar`, `funcion_anonima`)
Visitor y motor de procesamiento	`src/visitante_evaluador.py`, `src/operaciones.py`, `src/tabla.py`
Tabla de símbolos	Diccionario `memoria` del Visitor, ahora con tablas
Documentación de reglas semánticas	Documento *Documentación de las reglas semánticas*
Pruebas unitarias y de integración	`test_terra.py` (100), `test_corte2.py` (53) y `test_biblioteca.py` (54)
Mínimo tres programas completos	Once programas, del `07` al `18`
Evidencia de transformación y exportación	Secciones 7 y 8, y la carpeta `salidas/`
Repositorio actualizado	`github.com/yslen2421/terra-corte-2`
Producto: datos reales y tabla resumen exportada	`13_caso_owid.terra` y `salidas/owid_*.csv`

12. Limitaciones y trabajo siguiente

Limitaciones actuales:

- `ordenar` recibe una sola columna; para ordenar por varias se aplica varias veces, aprovechando que el orden es estable.
- Las tablas se guardan completas en memoria, lo que limita el tamaño de archivo que se puede procesar.
- Las funciones anónimas tienen una sola expresión como cuerpo; para varias sentencias se usa una función con nombre.
- `graficar` se reconoce sintácticamente, pero su ejecución corresponde al tercer corte.

Tercera entrega. Generar al menos cuatro tipos de gráfica (barras, líneas, histograma y dispersión) exportadas a PNG sin librerías de visualización, dejar la interfaz de línea de comandos definitiva y completar el caso de estudio final sobre emisiones y energía.

13. Reproducción

```
git clone https://github.com/yslen2421/terra-corte-2.git
cd terra-corte-2

python3 -m venv venv
source venv/bin/activate
pip install -r requisitos.txt

python src/terra.py ejemplos/10_agrupacion.terra
```

```
python src/terra.py ejemplos/13_caso_owid.terra
pytest tests/ -q
```

Requisitos: Python 3.11 o superior. Java solo es necesario si se modifica la gramática y hay que regenerar el parser con `./generar.sh`.